Stagewise Multi-Agent Plugin Development

Umfassende Anleitung zur Entwicklung von Custom Plugins für Multi-Agent-Koordination zwischen Cursor, Windsurf und anderen AI-Code-Editoren

Inhaltsverzeichnis

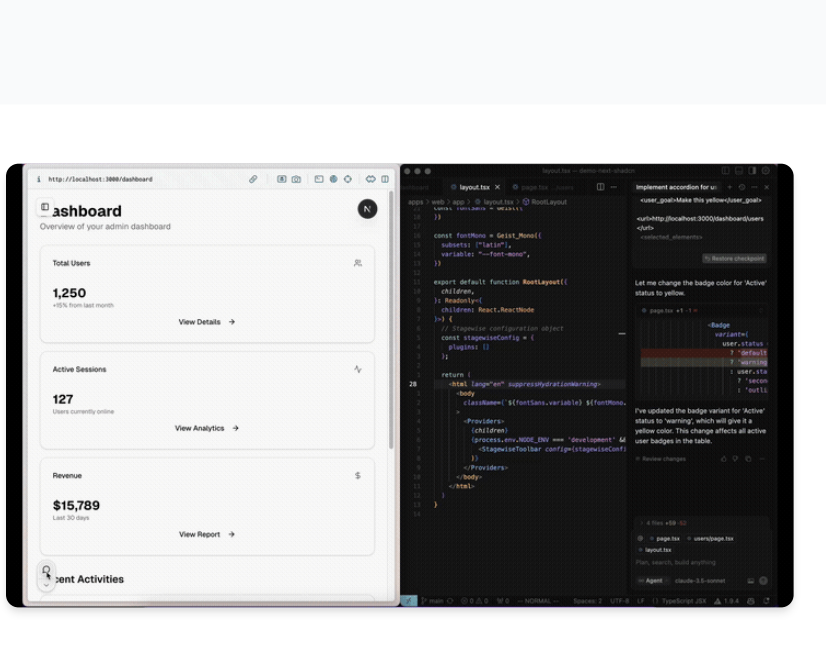
1. Stagewise Überblick
2. Plugin-Architektur
3. TypeScript Interfaces
4. Multi-Agent-Koordination
5. Implementierung
6. Kommunikation
7. Praktische Beispiele
8. Deployment & Testing

Stagewise Überblick

Was ist Stagewise?

Stagewise ist ein innovatives Open-Source-Tool, das eine Brücke zwischen Browser und AI-Code-Editoren schlägt. Es ermöglicht direkte Interaktion mit UI-Elementen und automatische Weiterleitung an AI-Agenten.

- ✓ Browser-Toolbar Integration
- ✓ VS Code Extension Kompatibilität
- ✓ DOM-Element-Auswahl
- ✓ Screenshot & Metadata-Übertragung



Browser-Integration

NPM-Paket für nahtlose Integration in Web-Anwendungen aller Frameworks

VS Code Extension

Kompatibel mit Cursor, Windsurf und anderen VS Code basierten Editoren

Plugin-System

Erweiterbare Architektur für Custom Plugins und Funktionalitäten

Plugin-Architektur

Grundlegende Architektur

Komponentenübersicht

Browser Toolbar	Frontend-Integration über NPM-Pakete
Communication Layer	WebSocket/Message-Passing zwischen Browser und IDE
IDE Extension	VS Code Extension für Cursor/Windsurf
AI Agents	Multiple AI-Agenten für verschiedene Aufgaben

Datenfluss

```
Browser UI
    ↓ (DOM Selection + Context)
Plugin System
    ↓ (Structured Message)
Communication Layer
    ↓ (Multi-Agent Broadcast)
IDE Extensions
    ↓ (Agent-specific Prompts)
AI Agents (Cursor, Windsurf, etc.)
    ↓ (Code Changes)
Shared Repository/Thread
```

TypeScript Interfaces

Erweiterte Plugin-Interfaces

Multi-Agent Plugin Interface

```typescript
// types/multi-agent.ts
export interface MultiAgentPlugin {
  name: string;
  description: string;
  version: string;
  supportedAgents: AgentType[];

  // Lifecycle Methods
  initialize(config: MultiAgentConfig): Promise<void>;
  destroy(): Promise<void>;

  // Core Functionality
  shortInfoForPrompt(context: ElementContext): string;
  processMessage(message: AgentMessage): Promise<ProcessedMessage>;

  // Multi-Agent Coordination
  broadcastToAgents(message: string, agents: AgentType[]): Promise<void>;
  createSharedThread(threadConfig: ThreadConfig): Promise<string>;

  // Action System
  actions: MultiAgentAction[];
  mcp?: MCPConfig;
}

export interface MultiAgentConfig {
  agents: AgentConfiguration[];
  communicationMode: 'websocket' | 'messagebus' | 'hybrid';
  threadManagement: ThreadManagementConfig;
  fallbackStrategy: FallbackStrategy;
}

export interface AgentConfiguration {
  type: AgentType;
  endpoint: string;
  capabilities: string[];
  priority: number;
  specialization: AgentSpecialization;
}

export type AgentType =
  | 'cursor'
  | 'windsurf'
  | 'copilot'
  | 'cline'
  | 'continue';
```

Thread Management Interface

```typescript
// types/thread-management.ts
export interface ThreadManager {
  createThread(config: ThreadConfig): Promise<Thread>;
  joinThread(threadId: string, agent: AgentType): Promise<void>;
  broadcastMessage(threadId: string, message: ThreadMessage): Promise<void>;
  syncState(threadId: string): Promise<ThreadState>;

  // Event Handlers
  onMessageReceived: (callback: MessageHandler) => void;
  onAgentJoined: (callback: AgentJoinHandler) => void;
  onStateChanged: (callback: StateChangeHandler) => void;
}

export interface Thread {
  id: string;
  title: string;
  participants: AgentParticipant[];
  messages: ThreadMessage[];
  sharedContext: SharedContext;
  metadata: ThreadMetadata;
}

export interface ThreadMessage {
  id: string;
  threadId: string;
  sender: AgentType | 'user';
  content: string;
  attachments: MessageAttachment[];
  timestamp: Date;
  messageType: 'prompt' | 'response' | 'context' | 'system';
}
```

Communication Protocol Interface

```typescript
// types/communication.ts
export interface CommunicationProtocol {
  connect(): Promise<void>;
  disconnect(): Promise<void>;

  sendMessage(message: ProtocolMessage): Promise<void>;
  subscribe(channel: string, handler: MessageHandler): void;
  unsubscribe(channel: string): void;

  // Multi-Agent Broadcasting
  broadcast(message: BroadcastMessage): Promise<void>;
  createChannel(channelConfig: ChannelConfig): Promise<string>;
}

export interface ProtocolMessage {
  type: MessageType;
  source: AgentType | 'browser';
  target: AgentType | 'all';
  payload: MessagePayload;
  metadata: MessageMetadata;
}

export interface BroadcastMessage extends ProtocolMessage {
  targets: AgentType[];
  priority: 'low' | 'normal' | 'high' | 'urgent';
  requiresAck: boolean;
}
```

Multi-Agent-Koordination

Koordinationsstrategien

Round-Robin Assignment
Aufgaben werden gleichmäßig zwischen verfügbaren Agenten verteilt

Specialization-Based Routing
Agenten werden basierend auf ihren Spezialisierungen ausgewählt

Collaborative Mode
Mehrere Agenten arbeiten gleichzeitig an der gleichen Aufgabe

Fallback Chain
Bei Fehlern wird automatisch zum nächsten verfügbaren Agent gewechselt

Agent-Spezialisierungen

UI/UX Specialist	Frontend-Design und Benutzeroberflächen
Backend Specialist	APIs, Datenbanken und Server-Logik
Testing Specialist	Test-Automation und Quality Assurance
Security Specialist	Sicherheitsaudits und Vulnerability-Checks

Implementierung

Multi-Agent Plugin Implementierung

1. Plugin-Klasse

```typescript
// plugins/multi-agent-coordinator.ts
import { MultiAgentPlugin, AgentType, ThreadManager } from '../types';

export class MultiAgentCoordinator implements MultiAgentPlugin {
  name = 'multi-agent-coordinator';
  description = 'Koordiniert mehrere AI-Agenten für kollaborative Entwicklung';
  version = '1.0.0';
  supportedAgents: AgentType[] = ['cursor', 'windsurf', 'copilot', 'cline'];

  private threadManager: ThreadManager;
  private communicationProtocol: CommunicationProtocol;
  private activeAgents: Map<AgentType, AgentStatus> = new Map();

  async initialize(config: MultiAgentConfig): Promise<void> {
    // Initialize communication channels
    this.communicationProtocol = new WebSocketProtocol(config.communicationMode);
    await this.communicationProtocol.connect();

    // Setup thread manager
    this.threadManager = new ThreadManager(config.threadManagement);

    // Register agents
    for (const agentConfig of config.agents) {
      await this.registerAgent(agentConfig);
    }

    // Setup event listeners
    this.setupEventHandlers();
  }

  async broadcastToAgents(message: string, agents: AgentType[]): Promise<void> {
    const broadcastMessage: BroadcastMessage = {
      type: 'prompt',
      source: 'browser',
      targets: agents,
      payload: { content: message },
      priority: 'normal',
      requiresAck: true,
      metadata: { timestamp: new Date(), messageId: generateId() }
    };

    await this.communicationProtocol.broadcast(broadcastMessage);
  }

  async createSharedThread(threadConfig: ThreadConfig): Promise<string> {
    const thread = await this.threadManager.createThread(threadConfig);

    // Invite specified agents to join
    for (const agentType of threadConfig.participants) {
      if (this.activeAgents.has(agentType)) {
        await this.threadManager.joinThread(thread.id, agentType);
      }
    }

    return thread.id;
  }

  shortInfoForPrompt(context: ElementContext): string {
    return `Multi-Agent Context:
- Selected Element: ${context.element.tagName} (${context.element.className})
- Available Agents: ${Array.from(this.activeAgents.keys()).join(', ')}
- Thread ID: ${context.threadId || 'new'}
- Task Type: ${this.detectTaskType(context)}`;
  }

  actions = [
    {
      name: 'Create Multi-Agent Thread',
      description: 'Erstellt einen neuen Thread für mehrere Agenten',
      execute: async (context: ElementContext) => {
        const threadId = await this.createSharedThread({
          title: `UI Änderung: ${context.element.tagName}`,
          participants: ['cursor', 'windsurf'],
          context: context
        });

        showNotification(`Thread ${threadId} erstellt`);
      }
    },
    {
      name: 'Broadcast to All Agents',
      description: 'Sendet Nachricht an alle verfügbaren Agenten',
      execute: async (prompt: string, context: ElementContext) => {
        const availableAgents = Array.from(this.activeAgents.keys());
        await this.broadcastToAgents(prompt, availableAgents);

        showNotification(`Nachricht an ${availableAgents.length} Agenten gesendet`);
      }
    },
    {
      name: 'Smart Agent Selection',
      description: 'Wählt automatisch den besten Agent für die Aufgabe',
      execute: async (prompt: string, context: ElementContext) => {
        const taskType = this.detectTaskType(context);
        const bestAgent = this.selectBestAgent(taskType);

        await this.broadcastToAgents(prompt, [bestAgent]);
        showNotification(`Aufgabe an ${bestAgent} (${taskType}) delegiert`);
      }
    }
  ];
}
```

2. WebSocket Communication Protocol

```typescript
// communication/websocket-protocol.ts
export class WebSocketProtocol implements CommunicationProtocol {
  private ws: WebSocket | null = null;
  private channels: Map<string, Set<MessageHandler>> = new Map();
  private reconnectAttempts = 0;
  private maxReconnectAttempts = 5;

  async connect(): Promise<void> {
    return new Promise((resolve, reject) => {
      try {
        // Connect to local WebSocket server (running alongside Stagewise extension)
        this.ws = new WebSocket('ws://localhost:3892/stagewise-multi-agent');

        this.ws.onopen = () => {
          console.log('Multi-Agent WebSocket connected');
          this.reconnectAttempts = 0;
          resolve();
        };

        this.ws.onmessage = (event) => {
          const message: ProtocolMessage = JSON.parse(event.data);
          this.handleIncomingMessage(message);
        };

        this.ws.onclose = () => {
          console.log('WebSocket connection closed');
          this.attemptReconnect();
        };

        this.ws.onerror = (error) => {
          console.error('WebSocket error:', error);
          reject(error);
        };
      } catch (error) {
        reject(error);
      }
    });
  }

  async sendMessage(message: ProtocolMessage): Promise<void> {
    if (this.ws?.readyState === WebSocket.OPEN) {
      this.ws.send(JSON.stringify(message));
    } else {
      throw new Error('WebSocket not connected');
    }
  }

  async broadcast(message: BroadcastMessage): Promise<void> {
    // Add routing information for multi-agent broadcasting
    const routedMessage: ProtocolMessage = {
      ...message,
      type: 'multi_agent_broadcast',
      metadata: {
        ...message.metadata,
        routingInfo: {
          targets: message.targets,
          priority: message.priority,
          requiresAck: message.requiresAck
        }
      }
    };

    await this.sendMessage(routedMessage);
  }

  subscribe(channel: string, handler: MessageHandler): void {
    if (!this.channels.has(channel)) {
      this.channels.set(channel, new Set());
    }
    this.channels.get(channel)!.add(handler);
  }

  private handleIncomingMessage(message: ProtocolMessage): void {
    // Route message to appropriate channel handlers
    const channelName = `${message.source}_${message.type}`;
    const handlers = this.channels.get(channelName);

    if (handlers) {
      handlers.forEach(handler => handler(message));
    }

    // Handle global broadcasts
    const globalHandlers = this.channels.get('global');
    if (globalHandlers) {
      globalHandlers.forEach(handler => handler(message));
    }
  }

  private async attemptReconnect(): Promise<void> {
    if (this.reconnectAttempts < this.maxReconnectAttempts) {
      this.reconnectAttempts++;
      console.log(`Attempting reconnect ${this.reconnectAttempts}/${this.maxReconnectAttempts}`);

      setTimeout(() => {
        this.connect().catch(console.error);
      }, 2000 * this.reconnectAttempts);
    }
  }
}
```

Thread Management & Kommunikation

Thread Manager Implementation

```typescript
// thread/thread-manager.ts
export class ThreadManager {
  private threads: Map<string, Thread> = new Map();
  private eventEmitter: EventEmitter = new EventEmitter();

  async createThread(config: ThreadConfig): Promise<Thread> {
    const thread: Thread = {
      id: generateId(),
      title: config.title,
      participants: [],
      messages: [],
      sharedContext: {
        selectedElement: config.context?.element,
        projectFiles: await this.getRelevantFiles(config.context),
        screenshots: config.context?.screenshots || []
      },
      metadata: {
        created: new Date(),
        lastActivity: new Date(),
        status: 'active'
      }
    };

    this.threads.set(thread.id, thread);
    this.eventEmitter.emit('threadCreated', thread);

    return thread;
  }

  async joinThread(threadId: string, agent: AgentType): Promise<void> {
    const thread = this.threads.get(threadId);
    if (!thread) {
      throw new Error(`Thread ${threadId} not found`);
    }

    const participant: AgentParticipant = {
      agent,
      joinedAt: new Date(),
      status: 'active',
      capabilities: this.getAgentCapabilities(agent)
    };

    thread.participants.push(participant);
    thread.metadata.lastActivity = new Date();

    // Notify all participants about new member
    await this.broadcastMessage(threadId, {
      id: generateId(),
      threadId,
      sender: 'system',
      content: `${agent} hat den Thread beigetreten`,
      attachments: [],
      timestamp: new Date(),
      messageType: 'system'
    });

    this.eventEmitter.emit('agentJoined', { threadId, agent });
  }

  async broadcastMessage(threadId: string, message: ThreadMessage): Promise<void> {
    const thread = this.threads.get(threadId);
    if (!thread) {
      throw new Error(`Thread ${threadId} not found`);
    }

    thread.messages.push(message);
    thread.metadata.lastActivity = new Date();

    // Send to all active participants
    const activeParticipants = thread.participants.filter(p => p.status === 'active');

    for (const participant of activeParticipants) {
      if (participant.agent !== message.sender) {
        await this.sendToAgent(participant.agent, message);
      }
    }

    this.eventEmitter.emit('messageAdded', { threadId, message });
  }

  async syncState(threadId: string): Promise<ThreadState> {
    const thread = this.threads.get(threadId);
    if (!thread) {
      throw new Error(`Thread ${threadId} not found`);
    }

    // Gather current state from all participants
    const agentStates: Map<AgentType, AgentState> = new Map();

    for (const participant of thread.participants) {
      try {
        const state = await this.getAgentState(participant.agent);
        agentStates.set(participant.agent, state);
      } catch (error) {
        console.warn(`Failed to get state from ${participant.agent}:`, error);
      }
    }

    return {
      thread,
      agentStates,
      consensusState: this.calculateConsensus(agentStates),
      lastSync: new Date()
    };
  }

  private async sendToAgent(agent: AgentType, message: ThreadMessage): Promise<void> {
    const agentMessage: AgentMessage = {
      type: 'thread_message',
      content: message.content,
      threadId: message.threadId,
      context: {
        sender: message.sender,
        messageType: message.messageType,
        attachments: message.attachments
      }
    };

    // Route to appropriate agent endpoint
    switch (agent) {
      case 'cursor':
        await this.sendToCursor(agentMessage);
        break;
      case 'windsurf':
        await this.sendToWindsurf(agentMessage);
        break;
      case 'copilot':
        await this.sendToCopilot(agentMessage);
        break;
      default:
        console.warn(`Unknown agent type: ${agent}`);
    }
  }

  private async sendToCursor(message: AgentMessage): Promise<void> {
    // Implementation for Cursor communication
    // Uses VS Code extension API or direct WebSocket
    const cursorExtension = await vscode.extensions.getExtension('stagewise.stagewise-vscode-extension');
    if (cursorExtension?.isActive) {
      await cursorExtension.exports.sendMessage(message);
    }
  }

  private async sendToWindsurf(message: AgentMessage): Promise<void> {
    // Similar implementation for Windsurf
    // May use different communication protocol
  }
}
```

Praktische Beispiele

Beispiel 1: UI-Redesign mit mehreren Agenten

```typescript
// Szenario: Benutzer wählt eine Komponente aus und m
const redesignPlugin = {
  name: 'ui-redesign-coordinator',
  description: 'Koordiniert UI-Redesign zwischen mehr

  actions: [
    {
      name: 'Multi-Agent Redesign',
      execute: async (prompt: string, context: Element
        // 1. Thread für Redesign erstellen
        const threadId = await threadManager.createThr
          title: `UI Redesign: ${context.element.tagNa
          participants: ['cursor', 'windsurf'],
          context: context
        });

        // 2. Spezialisierte Prompts für jeden Agent
        const designPrompt = `Design-Experte: ${promp
          Fokus: Visuelle Verbesserungen, Accessibili
          Element: ${context.element.outerHTML}`;

        const implementationPrompt = `Implementation-
          Fokus: Clean Code, Performance, Maintainabi
          Element: ${context.element.outerHTML}`;

        // 3. Parallel an beide Agenten senden
        await Promise.all([
          broadcastToAgents(designPrompt, ['cursor'])
          broadcastToAgents(implementationPrompt, ['w
        ]);

        // 4. Ergebnisse koordinieren
        await coordinateResults(threadId);
      }
    }
  ]
};
```

Workflow: Ein Agent fokussiert sich auf Design-Aspekte, der andere auf technische Implementierung. Die Ergebnisse werden dann zusammengeführt.

Beispiel 2: Code Review mit Fallback-Kette

```typescript
// Szenario: Automatisches Code Review mit mehreren A
const codeReviewPlugin = {
  name: 'multi-agent-code-review',
  description: 'Führt Code Reviews mit mehreren Agente

  actions: [
    {
      name: 'Comprehensive Code Review',
      execute: async (context: ElementContext) => {
        const reviewers = [
          { agent: 'cursor', specialization: 'security
          { agent: 'windsurf', specialization: 'perfor
          { agent: 'copilot', specialization: 'best-pr
        ];

        const reviewResults = [];

        for (const reviewer of reviewers) {
          try {
            const result = await performReview(reviewe
            reviewResults.push(result);
          } catch (error) {
            console.warn(`${reviewer.agent} review fa
            continue;
          }
        }

        // Erstelle zusammengefassten Report
        const consolidatedReport = await consolidateR
        await displayReviewReport(consolidatedReport)
      }
    }
  ]
};
```

Workflow: Verschiedene Agenten führen spezialisierte Code Reviews durch. Bei Ausfall eines Agents übernimmt automatisch der nächste.

Beispiel 3: Kollaborative Feature-Entwicklung

```typescript
// Vollständige Implementierung einer Multi-Agent-Feature-Entwicklung
export class CollaborativeFeatureDevelopment {
  async developFeature(featureRequest: string, context: ElementContext): Promise<void> {
    // 1. Feature-Analyse und Aufgabenteilung
    const featureAnalysis = await this.analyzeFeature(featureRequest);
    const tasks = await this.breakDownIntoTasks(featureAnalysis);

    // 2. Thread für Feature-Entwicklung erstellen
    const threadId = await threadManager.createThread({
      title: `Feature: ${featureAnalysis.title}`,
      participants: ['cursor', 'windsurf', 'copilot'],
      context: {
        ...context,
        featureSpec: featureAnalysis,
        tasks: tasks
      }
    });

    // 3. Aufgaben verteilen
    const taskAssignments = await this.assignTasks(tasks, threadId);

    // 4. Parallele Entwicklung starten
    const developmentPromises = taskAssignments.map(assignment =>
      this.executeTask(assignment, threadId)
    );

    // 5. Synchronisation und Integration
    const results = await Promise.allSettled(developmentPromises);
    await this.integrateResults(results, threadId);

    // 6. Final review und Deployment
    await this.finalReview(threadId);
  }

  private async assignTasks(tasks: Task[], threadId: string): Promise<TaskAssignment[]> {
    const assignments: TaskAssignment[] = [];

    for (const task of tasks) {
      const bestAgent = await this.selectBestAgentForTask(task);
      assignments.push({
        task,
        agent: bestAgent,
        threadId,
        priority: task.priority,
        estimatedDuration: task.estimatedDuration
      });

      // Benachrichtige Agent über Aufgabe
      await threadManager.broadcastMessage(threadId, {
        id: generateId(),
        threadId,
        sender: 'system',
        content: `Task assigned to ${bestAgent}: ${task.description}`,
        attachments: [{ type: 'task_spec', data: task }],
        timestamp: new Date(),
        messageType: 'system'
      });
    }

    return assignments;
  }

  private async executeTask(assignment: TaskAssignment, threadId: string): Promise<TaskResult> {
    const { task, agent } = assignment;

    try {
      // Task-spezifischen Prompt erstellen
      const prompt = this.createTaskPrompt(task);

      // An spezifischen Agent senden
      await broadcastToAgents(prompt, [agent]);

      // Auf Completion warten
      const result = await this.waitForTaskCompletion(assignment);

      // Fortschritt an Thread melden
      await threadManager.broadcastMessage(threadId, {
        id: generateId(),
        threadId,
        sender: agent,
        content: `Task completed: ${task.title}`,
        attachments: [{ type: 'task_result', data: result }],
        timestamp: new Date(),
        messageType: 'response'
      });

      return result;
    } catch (error) {
      // Fehlerbehandlung und Fallback
      console.error(`Task execution failed for ${agent}:`, error);
      return await this.handleTaskFailure(assignment, threadId);
    }
  }
}
```

Deployment & Testing

Plugin-Setup und Integration

1. NPM Package Installation

```
npm install @stagewise/toolbar @stagewise/multi-agent-plugin
```

2. Plugin-Konfiguration

```typescript
// main.tsx
import { MultiAgentCoordinator } from '@stagewise/m

const multiAgentConfig = {
```

Testing Strategy

Unit Tests
Testen Sie einzelne Plugin-Komponenten isoliert

```
jest --testPathPattern=plugins/tests
```

Integration Tests
Testen Sie die Kommunikation zwischen Browser und IDEs

```
npm run test:integration
```

End-to-End Tests
Vollständige Workflows mit mehreren Agenten testen

```
playwright test e2e/multi-agent
```

Erstellt mit Genspark

```
  agents: [
    { type: 'cursor', endpoint: 'ws://localhost:389
    { type: 'windsurf', endpoint: 'ws://localhost:3
  ],
  threadManagement: { maxThreads: 10, autoCleanup:
};

const stagewiseConfig = {
  plugins: [new MultiAgentCoordinator(multiAgentCon
};
```

3. VS Code Extension Setup

Installieren Sie die Stagewise Extension in Cursor und Windsurf über den Extension Marketplace.

Performance Tests

WebSocket-Performance und Speicherverbrauch messen

```
npm run test:performance
```

Troubleshooting & Best Practices

Häufige Probleme

WebSocket-Verbindungsfehler
Stellen Sie sicher, dass alle IDE-Extensions aktiv sind und auf den richtigen Ports lauschen.

Message Delivery Failures
Implementieren Sie Retry-Mechanismen und Fallback-Strategien für kritische Nachrichten.

Thread Synchronisation
Verwenden Sie Timestamps und Sequence Numbers für konsistente Thread-Zustände.

Best Practices

Graceful Degradation
Plugin sollte auch mit nur einem verfügbaren Agent funktionieren.

Context Preservation
Speichern Sie wichtige Kontext-Informationen persistent zwischen Sessions.

User Feedback
Implementieren Sie klare Statusanzeigen und Fortschritts-Updates.

Fazit und Ausblick

Mit Custom Plugins für Stagewise können Sie eine leistungsstarke Multi-Agent-Entwicklungsumgebung schaffen, die die Stärken verschiedener AI-Code-Editoren kombiniert und kollaborative Entwicklungsprozesse ermöglicht.

Innovation
Neuartige Ansätze für AI-Agent-Koordination

Kollaboration
Nahtlose Zusammenarbeit zwischen verschiedenen Tools

Effizienz
Deutlich verbesserte Entwicklungsgeschwindigkeit

Stagewise Multi-Agent Plugin Development Guide - Eine umfassende Anleitung für fortgeschrittene AI-gestützte Entwicklung